

Phase 1: Conception

Hotel Booking Chatbot for **Biol Hotel**

IU International University of Applied Sciences

Course: Project: AI Use Case (CSEMAIPAIUC01), Task 1

Student: **Biol, Egemen**, Matriculation 10254279

1. Value proposition

Using the value-proposition canvas from Lecture 01:

For leisure and business travellers who want to book a hotel room quickly on their phone,
Dissatisfied with long multi-step web forms that require account creation and feel clumsy on a small screen,
Due to the lack of a single channel that handles the booking and answers basic questions about the hotel and the city,
Biol Hotel offers **Aria**, a chat-based booking assistant,
That provides a guided conversation, an instant booking reference, a clear total price, an in-chat card form, a personal receipt page, an admin dashboard for the hotel, and a 24/7 service that does not need an account.

2. Aims, target audience and MVP scope

Aims. The main aim of the chatbot is to collect the information needed to make a hotel reservation through a short conversation. The assignment requires three mandatory fields: name, time period, and number of guests. Payment and breakfast are added as optional extensions. A second aim is to answer simple questions about the hotel and about Istanbul, and to give the hotel staff a small admin view of the bookings.

Minimum Viable Segment (MVS). English-speaking leisure and business travellers, most of them on a phone. They can be reached through one channel: a web page (also embedded in a small iOS app).

Minimum Viable Product (MVP). A static web app with a guided booking dialogue covering eight slots (name, dates, room type, guests, breakfast, payment, payment_card, confirm), 16 concierge Q&A intents, a fallback for unrelated questions, a calendar widget that shows which dates are full, an in-chat card form, a personal receipt page, and a password-protected admin page for the hotel.

3. Structure and process rationale

The dialogue is built as a **slot-filling state machine** rather than a free-form language model. The booking domain is small and the set of fields is fixed, so a simple state machine fits well. It is also easier to read and grade: the whole dialogue graph is visible in the source code, and there is no risk of the model inventing a wrong reservation.

The system is built as a **static web app**, hosted on GitHub Pages. This was chosen so the project can be opened in a browser without any installation, which makes grading easier. The iOS app is a small WKWebView wrapper around the same site, so a push to the repository updates the app on the next launch with no rebuild.

4. Frameworks, tools and trade-offs

	Rasa	Dialogflow	Custom (chosen)
Hosting	Self-host server	Google Cloud	GitHub Pages (free)
Setup at grading time	Heavy install (broken on Py 3.14)	Google account + internet	Open the URL

NLU	Built-in pipeline (DIET, etc.)	Hosted neural model	Bag-of-words + cosine + regex
Dialog	Rule + ML stories	Contexts / fulfilment	Slot-filling FSM (8 slots)
Cost / month	Server costs	Per-call API bill	EUR 0 (Pages free tier)
Privacy	On your server	PII to Google	All client-side
iOS path	Backend + API	Backend + API	Same site in WKWebView
Auditable	Yes	Partly (cloud config)	Yes (every line in repo)

Decision. The custom static stack was chosen because it can be opened by the grader without any installation, all data stays on the device, and the iOS app updates automatically on every push to the repository. The trade-off is that a hand-written classifier does not generalise to paraphrases as well as a neural model, but the booking domain is small and the bot offers a fallback menu when a sentence does not match any pattern.

5. Information captured

Mandatory (assignment spec): guest name, stay period (check-in and check-out), number of guests.

Optional extensions implemented: room type (Standard, Deluxe, King Suite, Bosphorus Suite at EUR 120, 180, 280 and 420 per night), breakfast option (+EUR 10 per guest per night), payment method (credit card, debit card, or pay at the hotel), and a card form for the two card options that asks the guest to click a Pay button. The card form only collects the data: it does not run a real charge.

6. UML: Use case diagram

The use cases are grouped by area: concierge (cream), booking (teal) and admin (rose). The Guest is the main actor for the booking and concierge cases, the CEO is the main actor for the admin dashboard, and the browser storage is a secondary actor that keeps the bookings (localStorage) and the personal receipt scope (sessionStorage).

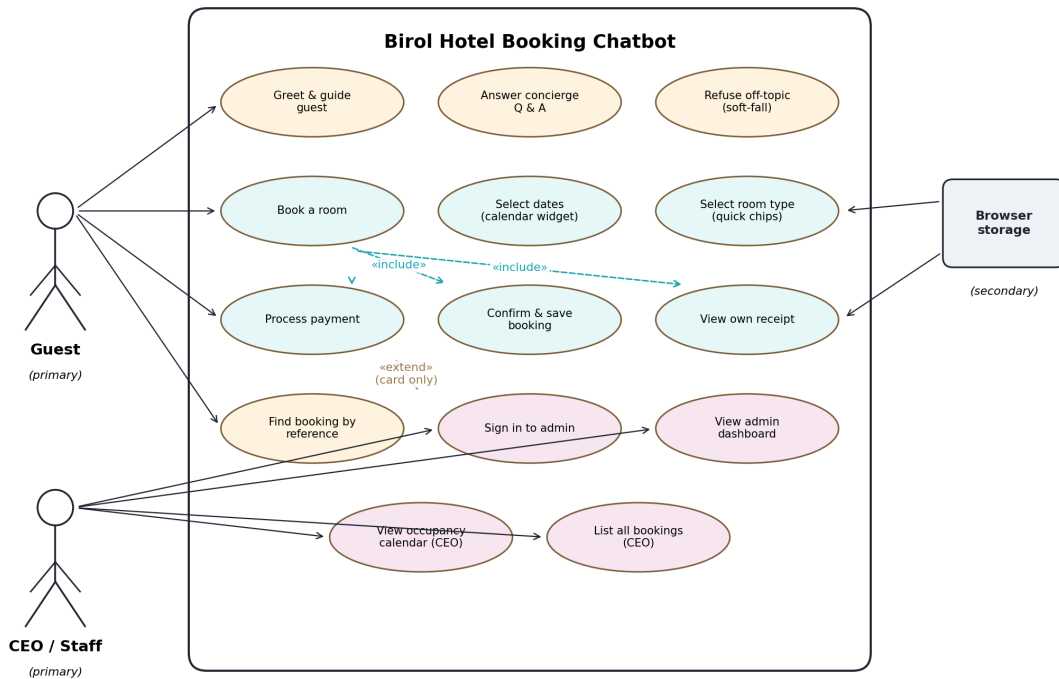


Figure 1. Use case diagram with 14 use cases grouped into three areas. Include and extend relationships are shown with dashed arrows.

7. UML: Sequence diagram (booking happy path)

The lifelines show the runtime layers: the web UI (chat.js + chat.html) sends each user message to the bot (bot.js, HotelBookingBot.respond()), which calls the intent classifier and the entity extractors. The booking store writes to localStorage on confirmation. The two interactive widgets (calendar and card form) are rendered by the UI in response to slot changes returned by the bot.

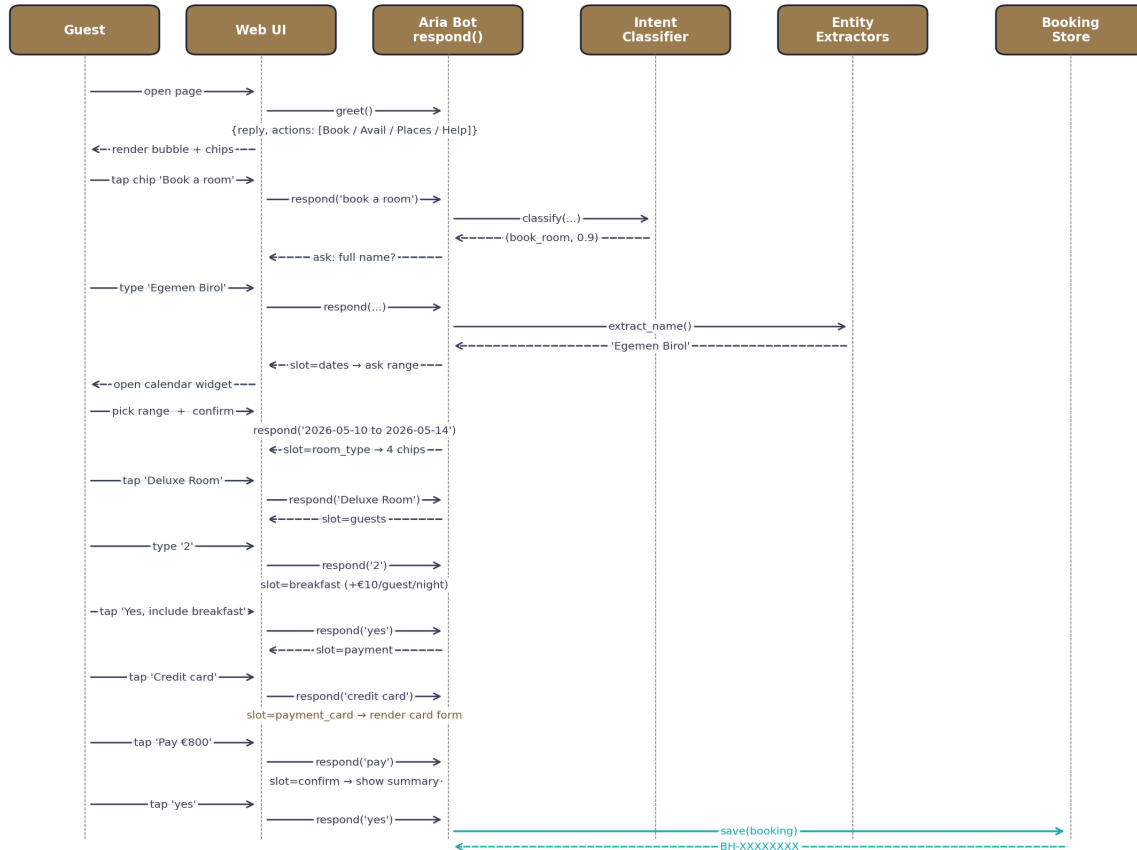


Figure 2. Sequence diagram for the booking happy path, including the calendar widget and the new card-payment step.

8. UML: State machine (slot machine)

The dialogue manager is a finite-state machine. A slot is filled by an entity extractor and the machine advances to the next missing slot. The three mandatory slots are highlighted in brass (per the assignment). Concierge intents are answered without changing the current slot, so the user can ask, for example, *where should I eat?* in the middle of a booking without losing their place. The payment_card state is reached only when the user picks credit or debit card; pay-at-hotel goes straight to confirm.

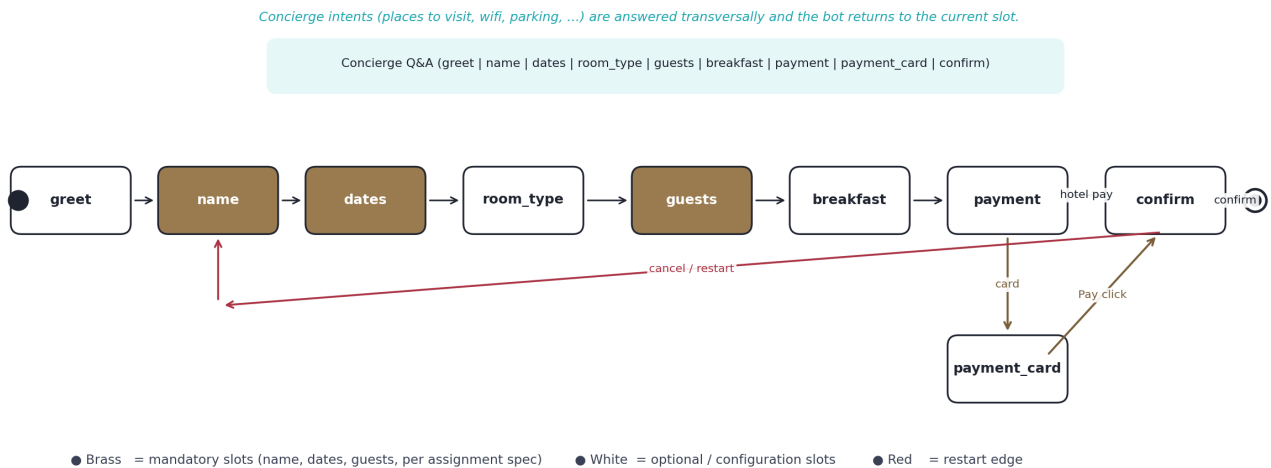


Figure 3. State machine for the booking conversation.

9. UML: Component and deployment

The whole system runs inside a single web container (the browser, or a WKWebView in the iOS app). The deployment loop is: push to GitHub, Pages rebuilds the site, the next page load downloads the new files. There is no server, no build step and no native binary to rebuild.

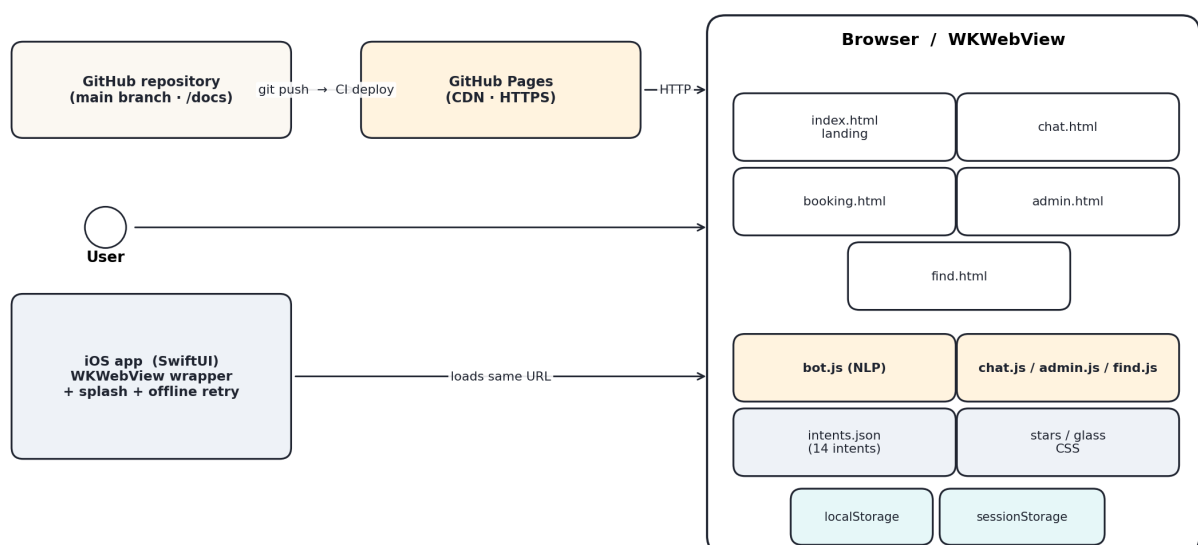


Figure 4. Component and deployment diagram.

10. Chatbot development workflow

The pipeline taught in Lecture 03/04 (define scope, list intents and entities, write training data, define the conversation flow, train and test, iterate, deploy) is followed in this project. Each stage produces a file in the repository.

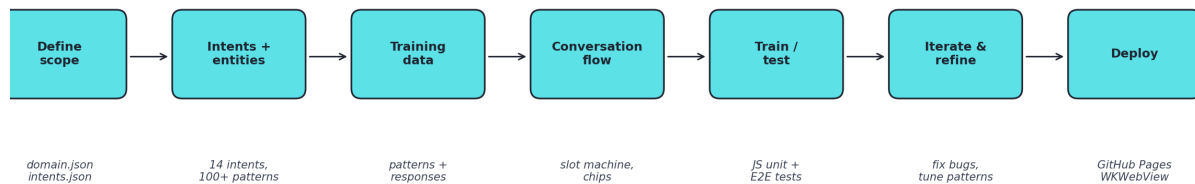


Figure 5. Project pipeline. The scope is in intents.json, the training data is in the patterns of each intent, the conversation flow is in chatbot.py / bot.js, the tests are in test_bot.py and test_web_bot.js, and deploy is to GitHub Pages.

11. Data and knowledge model

Intents (intents.json): 14 intents for the booking flow and general dialogue (greet, book_room, provide_name, provide_dates, provide_guests, breakfast_yes, breakfast_no, provide_payment, confirm_yes, confirm_no, goodbye, thanks, help, fallback) plus 16 concierge intents covering places_to_visit, food_recommendations, transport_airport, check_in_time, breakfast_hours, wifi, parking, amenities, cancellation_policy, pets, currency_info, tipping, language, hotel_contact, neighborhood, emergency, and room_types.

Domain entities: Booking (name, check-in, check-out, room type, room rate, guests, breakfast, payment, payment confirmed, booking id, status, total in EUR), RoomType (id, name, rate, short description) and Availability, which maps a date to one of three states: free, peak-full, or guest-booked.

Storage: localStorage holds the list of confirmed bookings (used by the admin page); sessionStorage holds the booking IDs of the current visit so the receipt page (booking.html) only shows the current guest's bookings. The admin login flag is also kept in sessionStorage.

12. Workplan and risk register

Stage	Output	Status
Conception (this document)	Concept text, UML diagrams, framework choice	this submission
Development	Working chatbot, 30 intents, slot machine, UI, iOS shell	complete
Reflection	10-slide development presentation	in progress
Finalization	2-page abstract, zip of all files, Phase 1 and 2 with corrections	pending feedback

Risk	Impact	Mitigation
Bag-of-words classifier misses paraphrases	User feels not understood	Fallback reply offers a chip menu (Help, Book a room) so the user can continue without typing
Stale browser cache after a new deploy	Users see old UI	?v= version on every asset URL plus no-cache meta tag on every HTML page
Receipt visible to a different guest	Privacy issue	Booking IDs are tagged in sessionStorage so the receipt page only shows the current visit

Admin password is checked in the browser	Easy to bypass	Documented as a soft gate, used only for the prototype
No backend, no shared view of all bookings	Architecture limit	Admin sees only the bookings made on the same device; a real backend (e.g. Firestore) could be added later

13. AI ethics and sustainability

Privacy. No personal data leaves the user's device. There are no analytics, no third-party trackers and no server-side logs. The NLU runs in the browser, and the chat transcript plus the booking are kept in the browser's local storage. The user can clear the data by clearing site data in the browser settings.

Inclusivity. Quick-reply chips reduce typing for users who are not comfortable writing in English. The shooting-star background respects the prefers-reduced-motion setting and stops animating for users who request reduced motion. All controls are reachable with the keyboard and have ARIA labels.

Transparency. The card form is clearly labelled as a demo ('Demo only, your card is not actually charged.'). When a question is outside the chatbot's scope, the bot says so and points to the Help menu instead of guessing.

Sustainability. The project does not use a backend, so there are no idle servers. GitHub Pages is served from a content delivery network. Each page is below 50 kB before fonts, which makes the energy use per session very low compared to a typical hotel website.

Submitted for the Conception phase of the portfolio (IU CSEMAIPAIUC01). The Phase 2 development presentation and the Phase 3 abstract follow the same naming convention.